

Periféricos y Comunicaciones en Sistemas Embebidos

clase 7: 14/07/2026

Objetivo de la clase

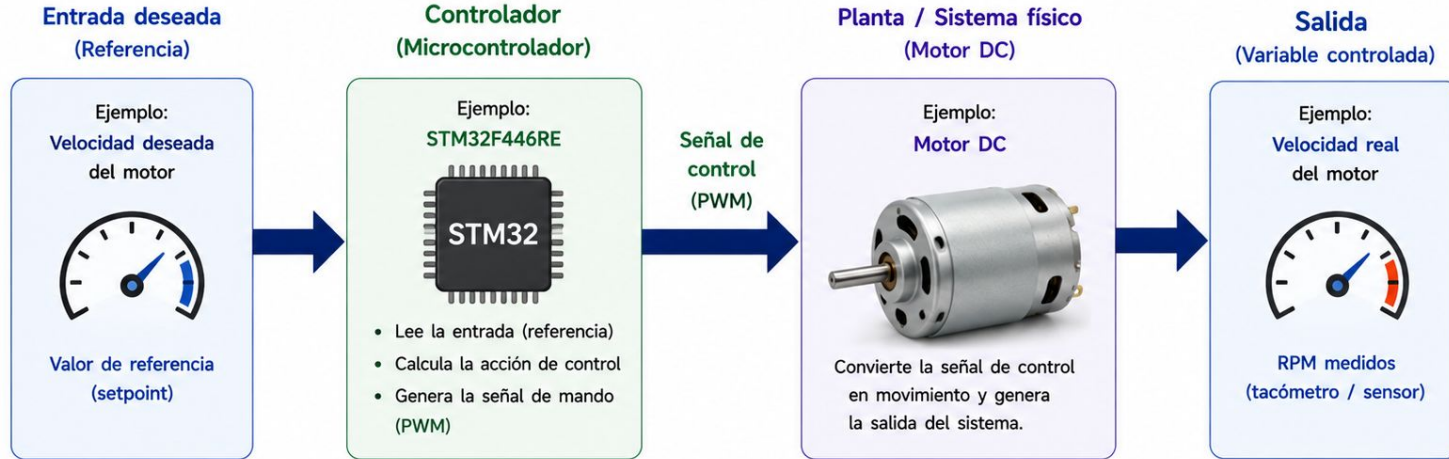
Comprender qué es un sistema de control, diferenciar lazo abierto y lazo cerrado, y analizar cómo la retroalimentación negativa permite reducir el error entre una referencia deseada y una salida medida.

Se abordará:

- Sistema de control.
- Referencia, controlador, actuador, planta y sensor.
- Lazo abierto y lazo cerrado.
- $\text{Error} = \text{referencia} - \text{salida medida}$.
- Retroalimentación negativa aplicada a un motor DC.

¿Qué es un sistema de control?

Un sistema de control guía a una planta (sistema físico) para que su salida se comporte como se desea.



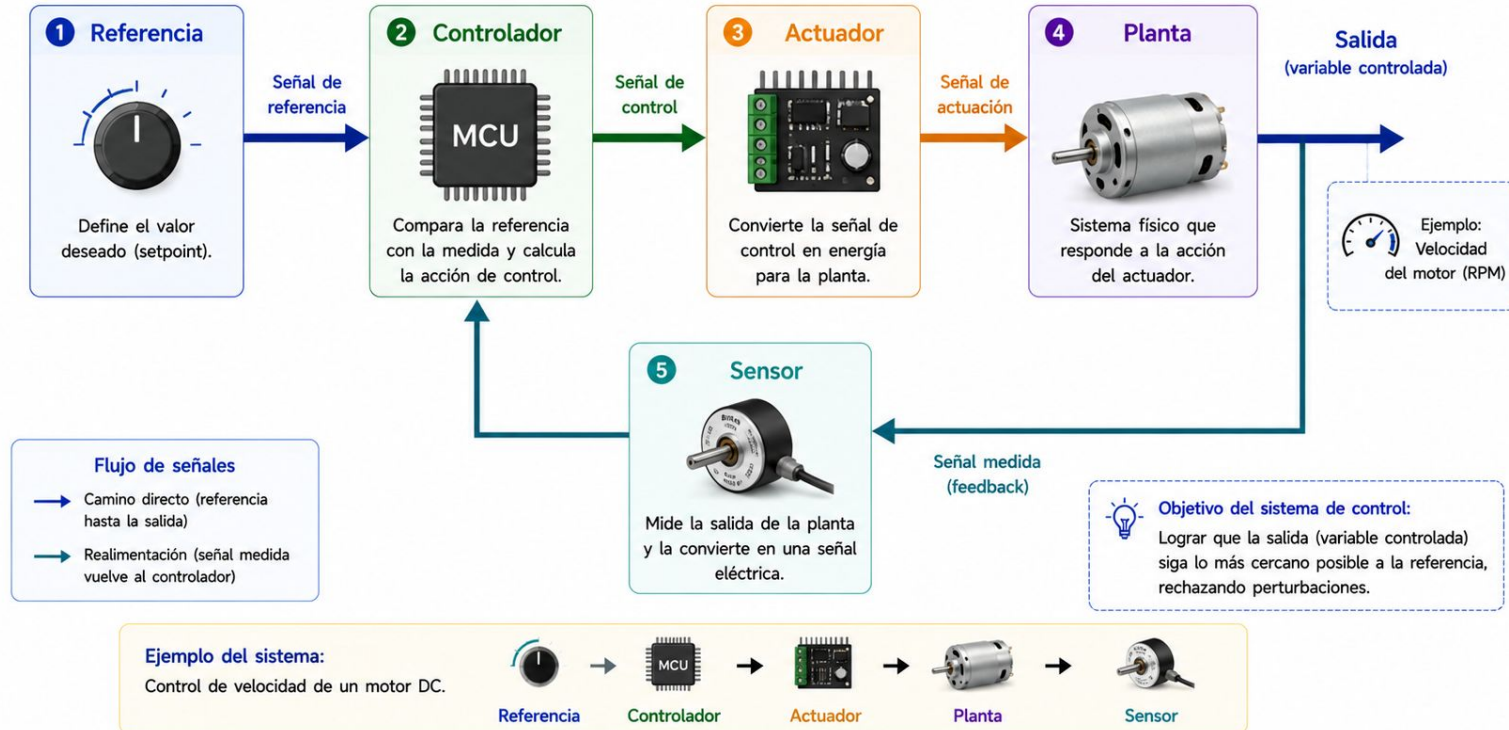
Objetivo del sistema de control:

Hacer que la salida real siga lo más cerca posible a la entrada deseada, a pesar de perturbaciones o cambios en la carga.



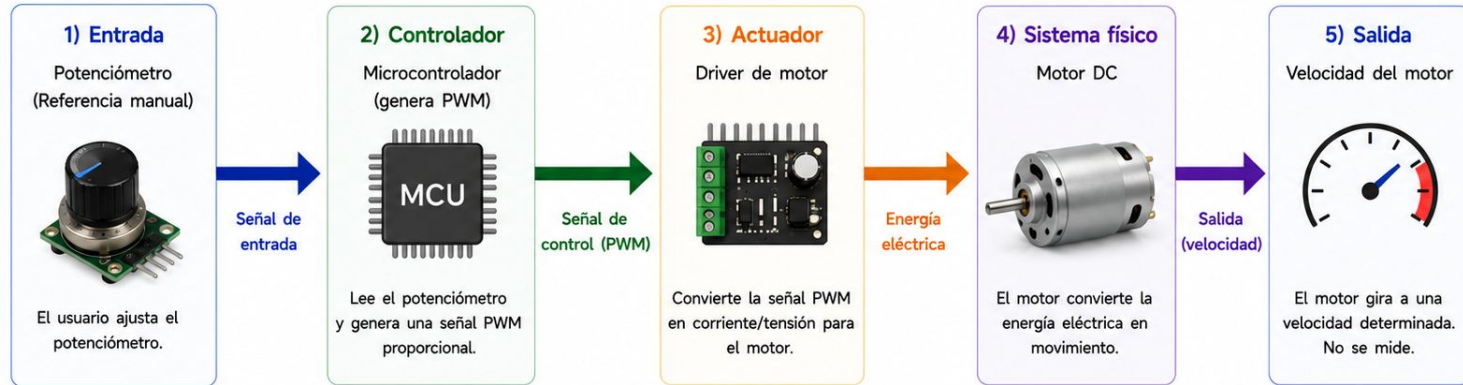
Ejemplo de funcionamiento: El microcontrolador ajusta el PWM aplicado al motor para lograr y mantener la velocidad deseada.

Elementos de un sistema de control



Control en lazo abierto

En un sistema de control en lazo abierto, la acción de control no depende de la salida.
No hay realimentación.



No se mide la salida

No hay sensor que informe la velocidad real al controlador.
Si cambia la carga o la tensión, la velocidad puede variar.



Ejemplo: El potenciometro fija el ciclo de trabajo (PWM) del motor.
Si aumenta la carga del motor, la velocidad puede disminuir, pero el sistema no lo detecta ni corrige.

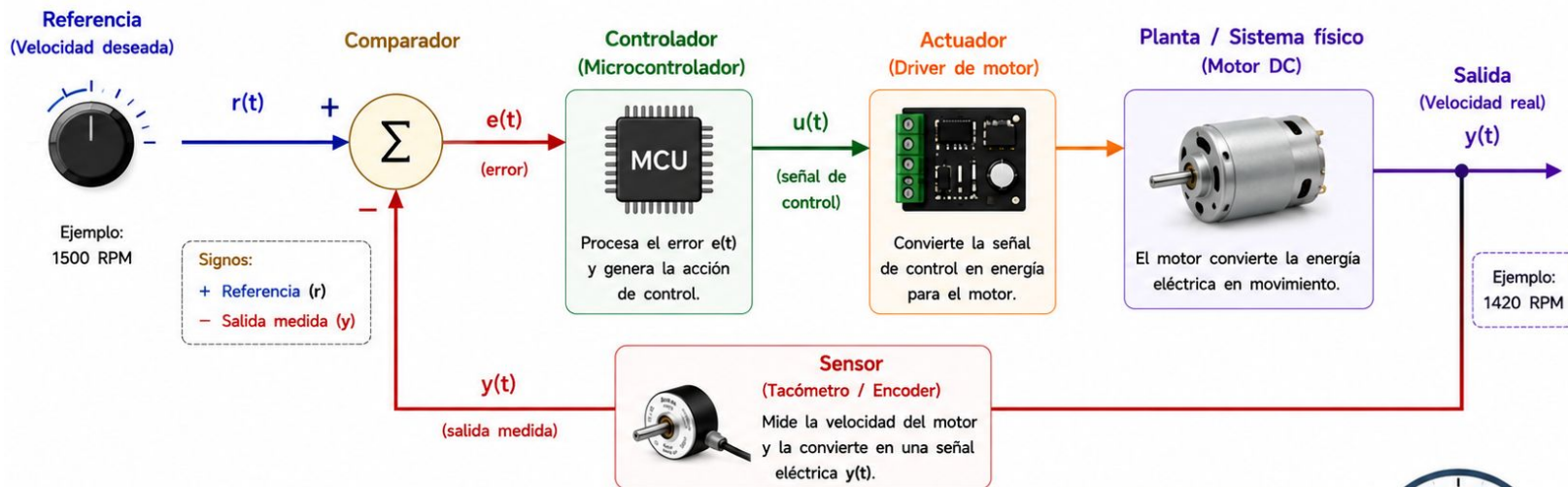
Control en lazo cerrado



Retroalimentación negativa

La salida medida se resta de la referencia. Si la salida es menor que la referencia, el sistema actúa para **aumentar** la velocidad, y **viceversa**.

$$\text{error} = \text{referencia} - \text{salida medida}$$



¿Cómo funciona?

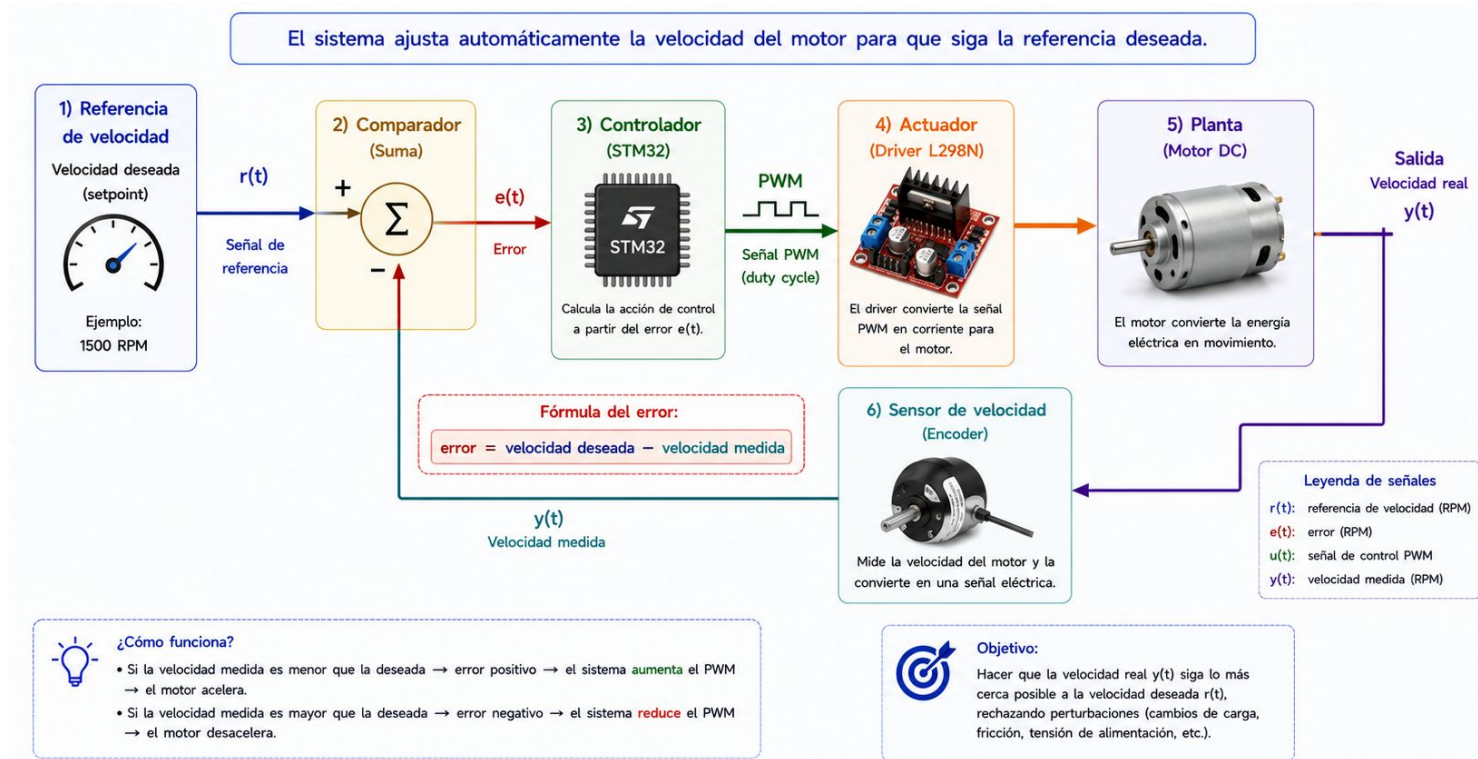
- Si la velocidad real es **menor** que la deseada → error positivo → el sistema **aumenta** la velocidad.
- Si la velocidad real es **mayor** que la deseada → error negativo → el sistema **reduce** la velocidad.

Objetivo:

Hacer que la salida medida $y(t)$ siga lo más cerca posible a la referencia $r(t)$, rechazando perturbaciones (cambios de carga, fricción, etc.).



Ejemplo: control de velocidad de motor DC



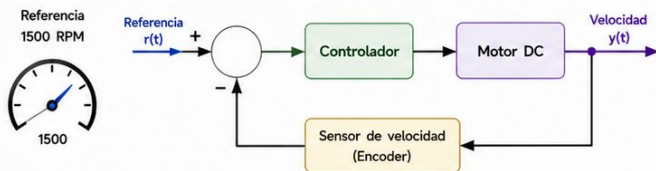
¿Por qué es útil la retroalimentación negativa?

Ventajas de la retroalimentación negativa

El sistema mide su salida, la compara con la referencia y corrige automáticamente los errores y cambios.

1 Reduce el error en estado estacionario

La salida se ajusta para ser lo más cercana posible a la referencia deseada.



Resultado: el error tiende a cero ($y(t) \approx r(t)$)

2 Rechaza perturbaciones y cambios de carga

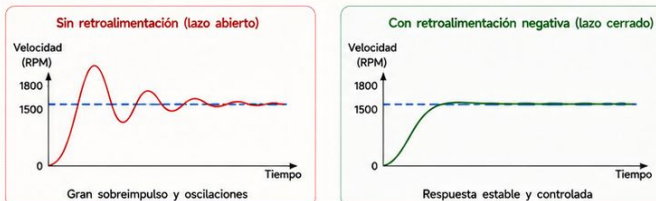
Si aparece una carga externa que tiende a reducir la velocidad, el sistema detecta el error y actúa para compensarlo.



Resultado: la velocidad vuelve a la referencia a pesar de la perturbación.

3 Mejora la estabilidad del sistema

La retroalimentación ayuda a evitar oscilaciones y comportamientos inestables, logrando una respuesta más suave y controlada.



4 Compensa variaciones del sistema

Si cambian las características del motor (temperatura, desgaste, tensión de alimentación), el sistema se adapta automáticamente.



En resumen: la retroalimentación negativa permite que el sistema sea más preciso, robusto y confiable frente a cambios y perturbaciones.

Otros ejemplos de retroalimentación negativa

En todos los casos, el sistema mide su salida, la compara con la referencia y actúa para reducir el error.

1 Dron estabilizando su inclinación



2 Tanque de agua controlando el nivel



3 Cámara ajustando el enfoque



La lógica es la misma: medir, comparar y corregir

4 Antena direccional corrigiendo su orientación



5 Robot seguidor de línea corrigiendo su trayectoria



Estos sistemas usan retroalimentación negativa para mantenerse estables, precisos y adaptarse a cambios.

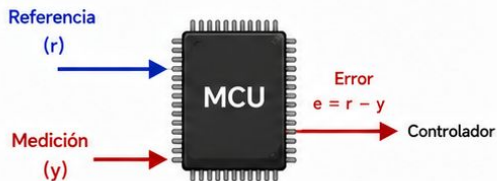
Bloque comparador: cálculo del error



¿Cómo se implementa el comparador?

1 Software en microcontrolador

El comparador se implementa por software dentro del microcontrolador.



Cálculo en el código:

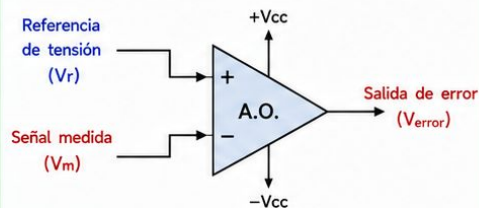
```
float error;  
error = referencia - medicion;
```

Ejemplos:

- Control de velocidad de un motor
- Control de ángulo de un dron
- Control de nivel de un tanque

2 Circuito analógico

El comparador se implementa con un circuito analógico, típicamente un amplificador operacional.



Función del comparador operacional:

$$V_{error} \propto (V_r - V_m)$$

Si $V_r > V_m \rightarrow V_{error}$ positivo
Si $V_r < V_m \rightarrow V_{error}$ negativo

Ejemplos:

- Comparación de tensiones (sensores analógicos)
- Control de posición con potenciómetros
- Control de corriente o nivel

3 Algoritmo digital

El comparador se implementa como un algoritmo que calcula el error de variables físicas.

Fórmula general:

$$\text{error} = \text{referencia} - \text{medición}$$



Ángulo

(dron, antena)

$$\text{error_ángulo} = \text{ángulo_deseado} - \text{ángulo_medido}$$



Posición

(ejes, robots)

$$\text{error_posición} = \text{posición_deseada} - \text{posición_medida}$$



Velocidad

(motores, vehículos)

$$\text{error_velocidad} = \text{velocidad_deseada} - \text{velocidad_medida}$$

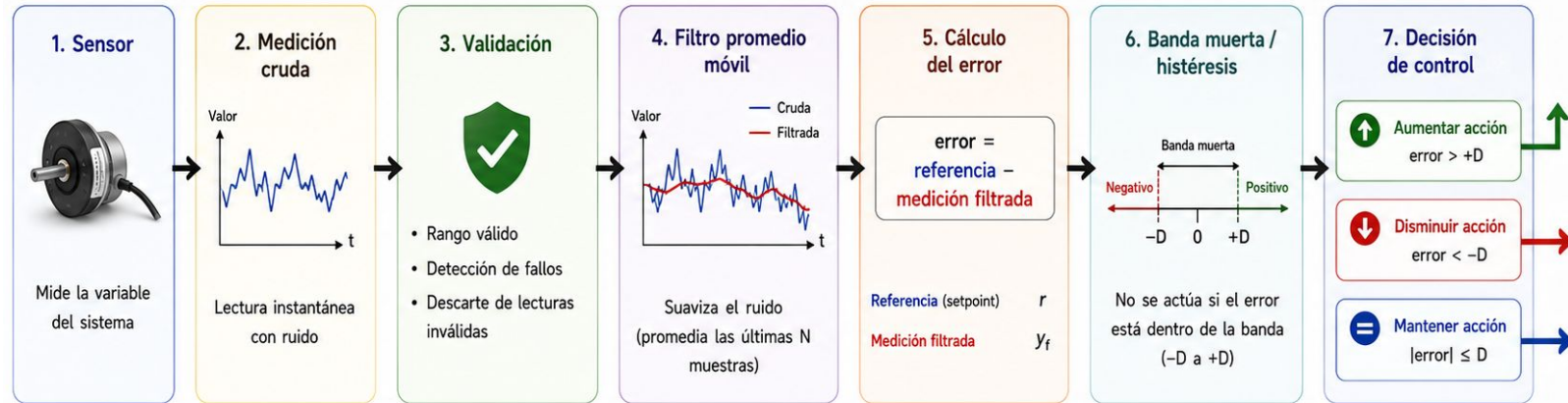
Ejemplos:

- Control PID de velocidad de motor
- Seguimiento de trayectoria en robots
- Control de orientación de antenas o satélites



El comparador calcula el error; el controlador decide la corrección.

Comparador implementado en software



No toda **diferencia** debe producir una **corrección inmediata**.

¿Para qué sirve?



Reduce el efecto del ruido



Evita correcciones innecesarias



Mejora la precisión del control



Aporta estabilidad y suavidad



Protege actuadores y prolonga vida útil

Comparador simple en software

Código C (ejemplo)

comparador.c

```
1 float referencia = 50.0f;  
2 float medicion = 47.0f;  
3  
4 float error = referencia - medicion;
```

Cálculo y decisión

error = 50.0 - 47.0

error = 3.0

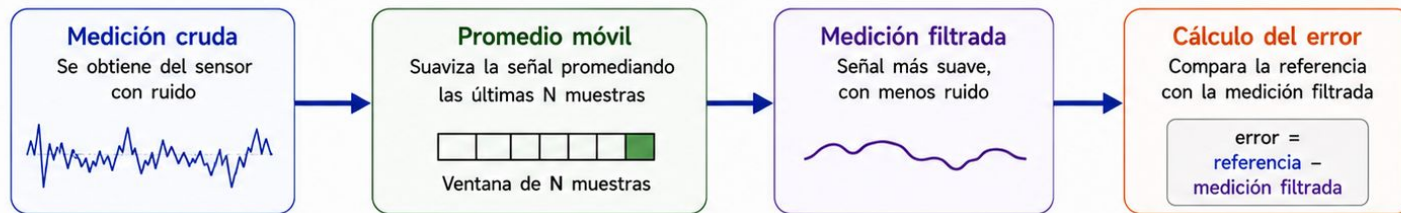


Corregir el sistema

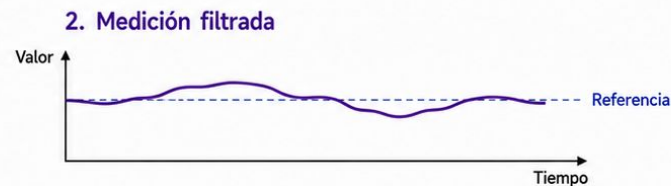
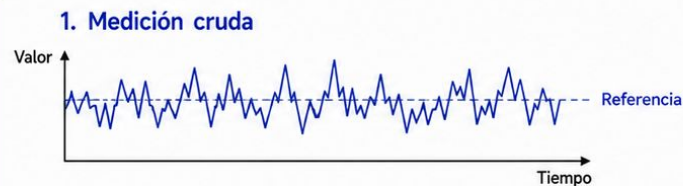


Responde ante cualquier diferencia,
incluso ruido.

Comparador con promedio móvil



```
1 float referencia = 50.0f;  
2 float medicion_cruda = leer_sensor();  
3 float medicion_filtrada = promedio_movil(medicion_cruda);  
4 float error = referencia - medicion_filtrada;
```



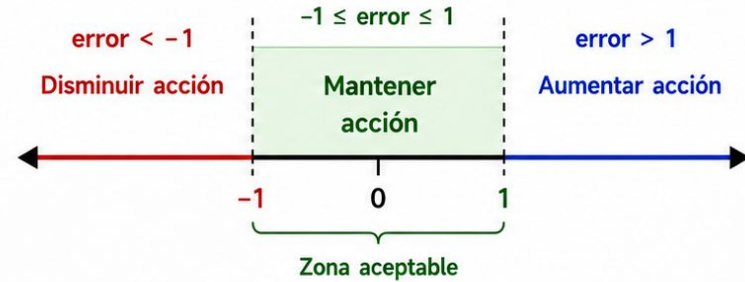
El filtro reduce el efecto del ruido antes de calcular el error.

Comparador con banda muerta

Codigo C (ejemplo)

```
1 float referencia = 50.0f;
2 float medicion_filtrada = promedio_movil(Leer_sensor());
3
4 float error = referencia - medicion_filtrada;
5 float banda_muerta = 1.0f;
6
7 if (error > banda_muerta)
8 {
9     aumentar_accion();
10 }
11 else if (error < -banda_muerta)
12 {
13     disminuir_accion();
14 }
15 else
16 {
17     mantener_accion();
18 }
```

Interpretación de la banda muerta



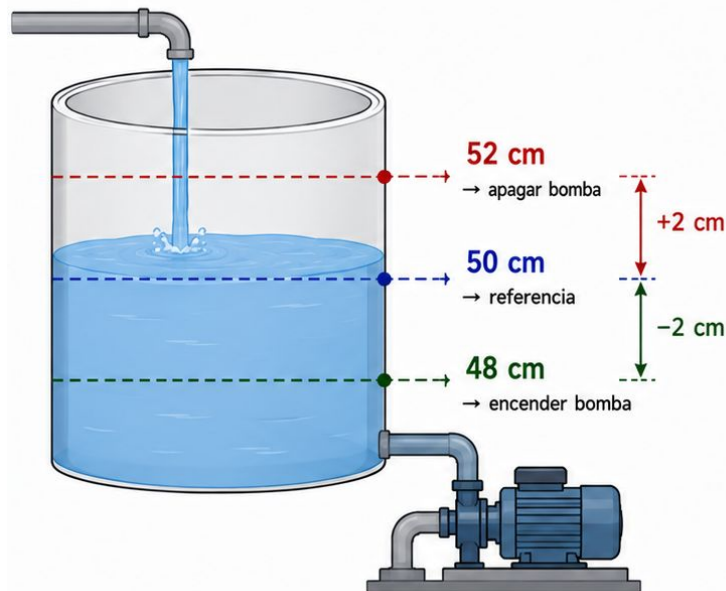
$\text{error} < -1$	La medición está por encima de lo deseado → Disminuir acción
$-1 \leq \text{error} \leq 1$	La medición está dentro de la zona aceptable → Mantener acción
$\text{error} > 1$	La medición está por debajo de lo deseado → Aumentar acción



No toda **diferencia** requiere una **corrección**.

Comparador con histéresis

Ejemplo: control de nivel de tanque de agua



```
1  #define NIVEL_REFERENCIA_CM  50.0f
2  #define HISTERESIS_CM        2.0f
3
4  static uint8_t bomba_encendida = 0;
5
6  void actualizar_control_nivel(float nivel_medido)
7  {
8      if (bomba_encendida == 0)
9      {
10         if (nivel_medido < (NIVEL_REFERENCIA_CM - HISTERESIS_CM))
11         {
12             bomba_encendida = 1;
13             encender_bomba();
14         }
15     }
16     else
17     {
18         if (nivel_medido > (NIVEL_REFERENCIA_CM + HISTERESIS_CM))
19         {
20             bomba_encendida = 0;
21             apagar_bomba();
22         }
23     }
24 }
```



La histéresis usa **dos umbrales** y **recuerda** el estado anterior.

Comparador: aumentar, disminuir o mantener



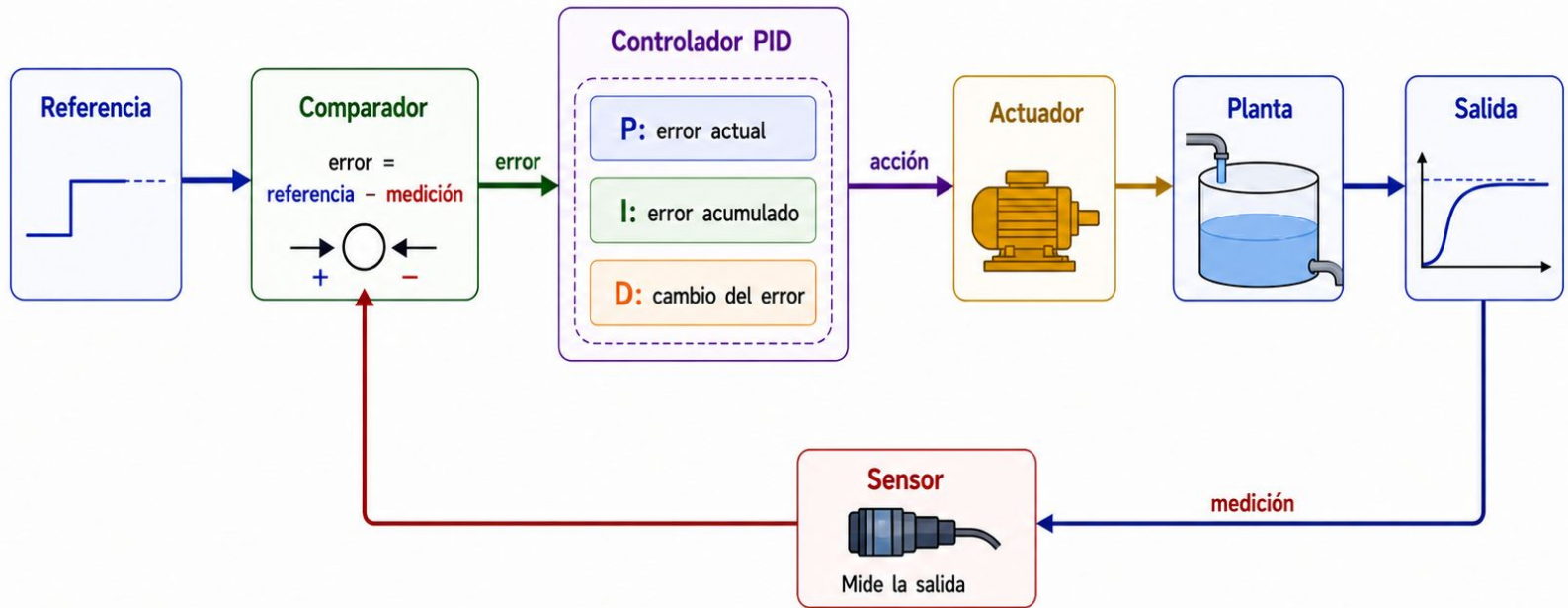
```
1 typedef enum
2 {
3     ACCION_DISMINUIR = -1,
4     ACCION_MANTENER = 0,
5     ACCION_AUMENTAR = 1
6 } accion_control_t;
```

```
7 accion_control_t comparar_con_banda_muerta(float referencia,
8                                             float medicion_filtrada,
9                                             float banda_muerta)
10 {
11     float error = referencia - medicion_filtrada;
12     if (error > banda_muerta)
13     {
14         return ACCION_AUMENTAR;
15     }
16     else if (error < -banda_muerta)
17     { return ACCION_DISMINUIR; }
18     else { return ACCION_MANTENER; }
19 }
```



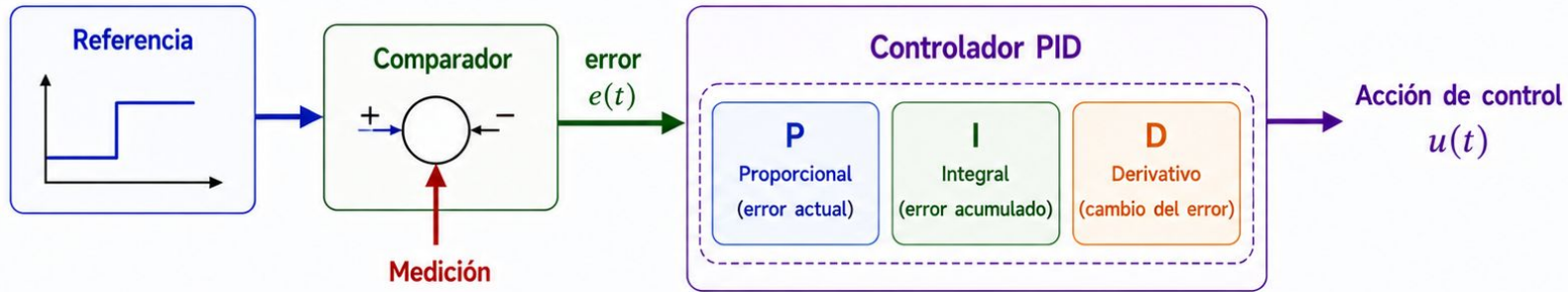
El comparador entrega una decisión estable al controlador.

Del comparador al controlador PID



El **comparador** calcula el **error**; el **PID** calcula la **acción de control**.

La ecuación del controlador PID



$$u(t) = K_p \cdot e(t) + K_i \cdot \int e(t) dt + K_d \cdot \frac{de(t)}{dt}$$

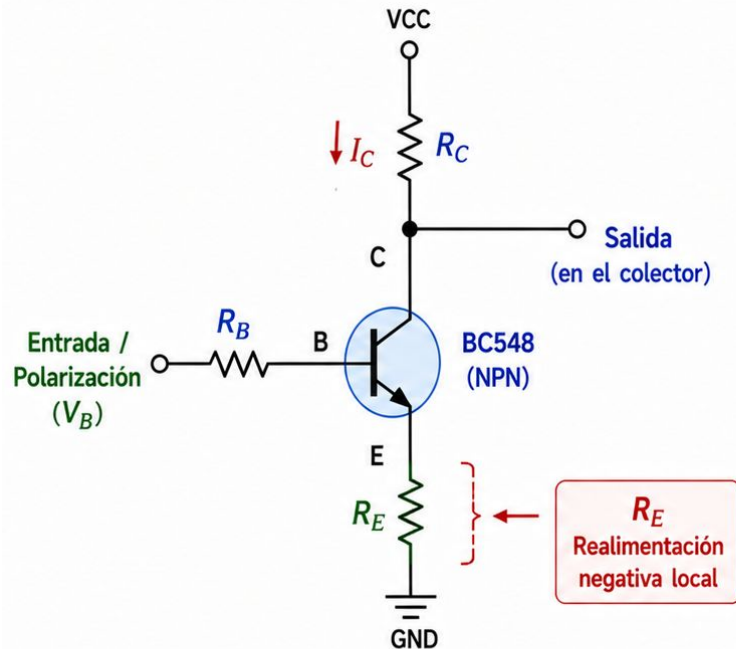
The equation is broken down into three components, each with a dashed arrow pointing to a descriptive box below:

- P: error actual**
Reacciona al error presente
- I: error acumulado**
Corrige errores persistentes
- D: cambio del error**
Anticipa tendencias y reduce oscilaciones



El **PID** transforma el **error** en una **acción de control**.

Retroalimentación negativa con transistor BC548



¿Cómo actúa la retroalimentación negativa?

- 1 Aumenta la **corriente** del transistor (I_C e I_E).
- 2 Aumenta la **tensión en R_E** .
- 3 Aumenta V_E (tensión en el emisor).
- 4 Disminuye V_{BE} (porque V_B se mantiene casi constante).
- 5 El transistor **conduce menos** (disminuye la corriente).

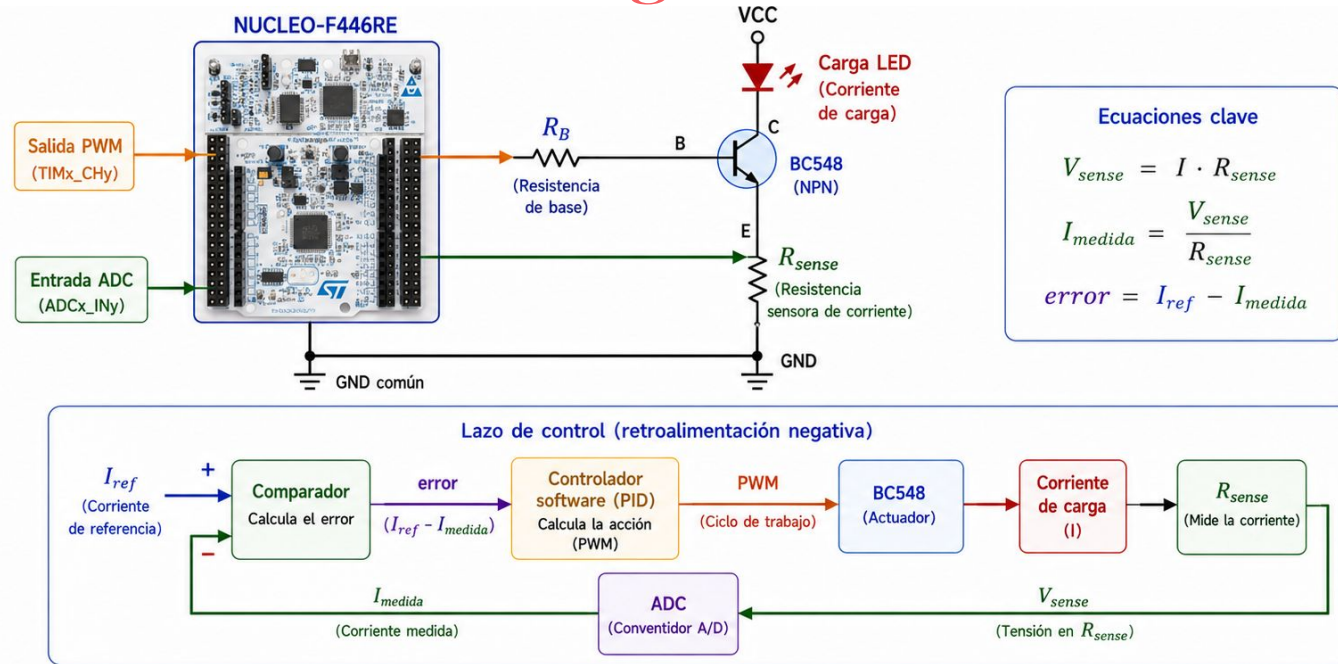
Expresiones clave

$$V_E = I_E \cdot R_E \quad \bigg| \quad V_{BE} = V_B - V_E$$



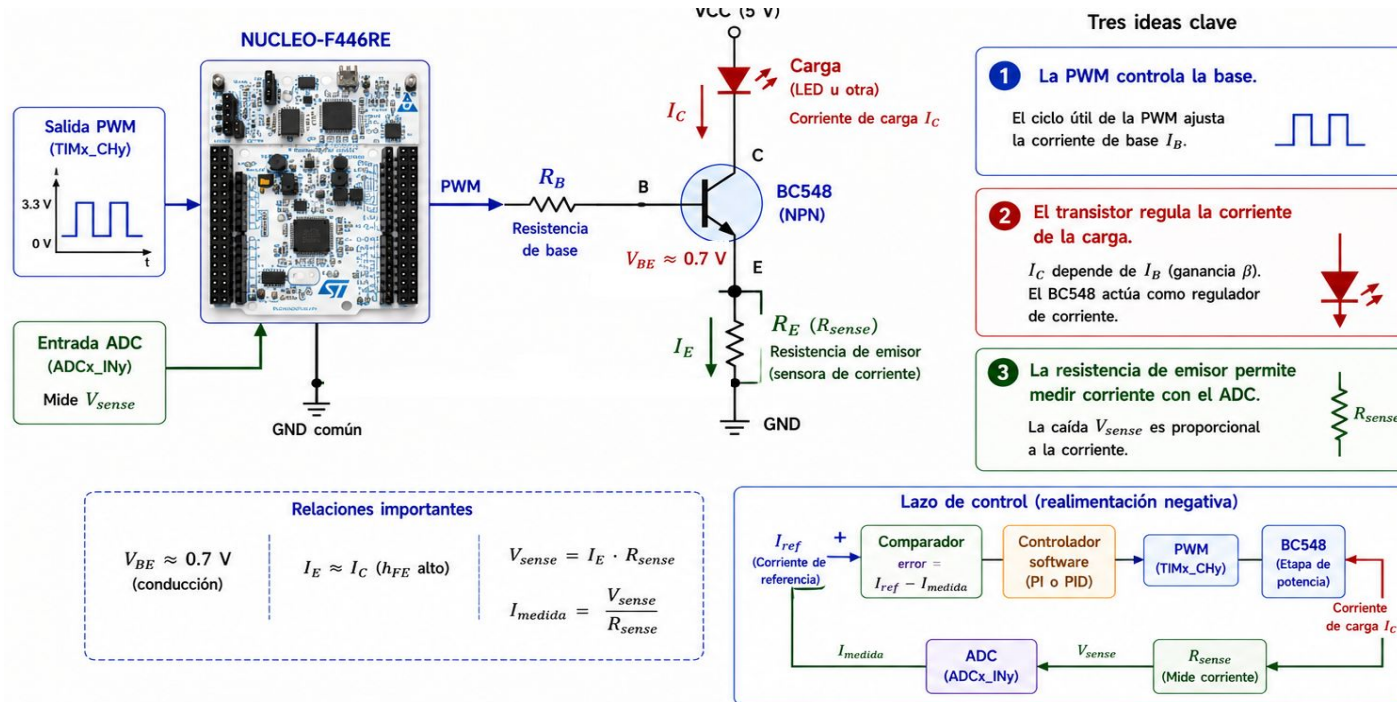
Si la **corriente aumenta**, el circuito genera un efecto que **la reduce**.

Actividad: control de corriente con retroalimentación negativa



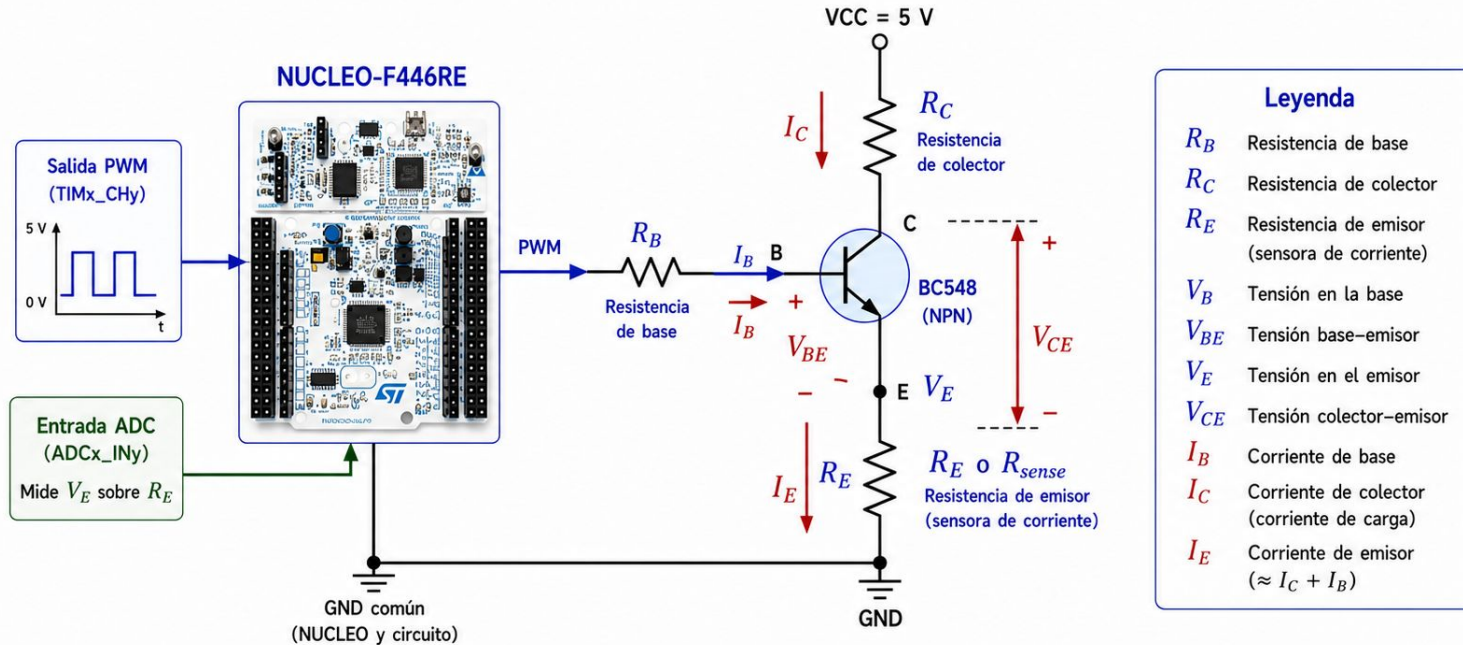
El ADC mide la realimentación; la PWM aplica la corrección.

Polarización del transistor BC548



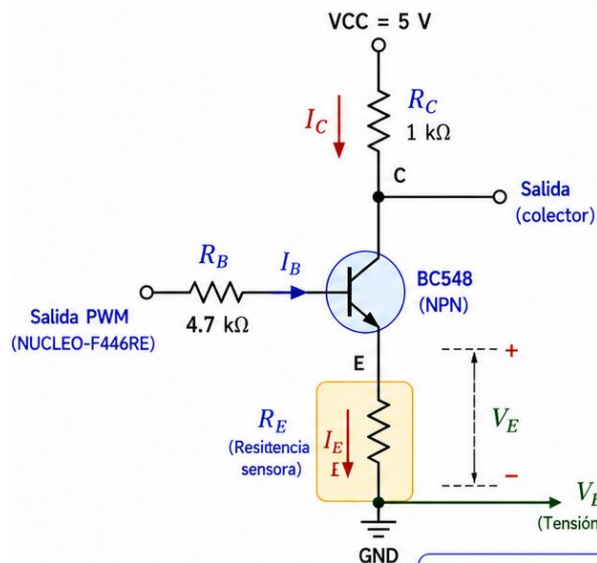
La polarización define un punto de operación seguro para el transistor.

Circuito de control de corriente con BC548



El ADC mide V_E ; a partir de V_E se estima la corriente.

Elección de R_E como resistencia sensora



Datos de diseño

- $I_C \approx 10 \text{ mA}$
- $I_E \approx I_C$ (en activa)
- $V_E = 1 \text{ V}$
- $R_E = \frac{V_E}{I_E}$

Cálculo de R_E

$$R_E = \frac{V_E}{I_E}$$

$$R_E = \frac{1 \text{ V}}{10 \text{ mA}}$$

$$R_E = 100 \Omega$$

Escala de funcionamiento

Corriente

$I_C (\approx I_E)$
fluye por el transistor



Genera tensión en R_E

$$V_E = I_E \cdot R_E$$



ADC mide V_E

y se estima la corriente

$$I_E = \frac{V_E}{R_E}$$

Ejemplo de verificación

Si $I_C = 10 \text{ mA} \Rightarrow V_E = 1 \text{ V}$

Si $I_C = 5 \text{ mA} \Rightarrow V_E = 0.5 \text{ V}$

Rango ADC (ejemplo)

Con ADC de 12 bits y $V_{ref} = 3.3 \text{ V}$

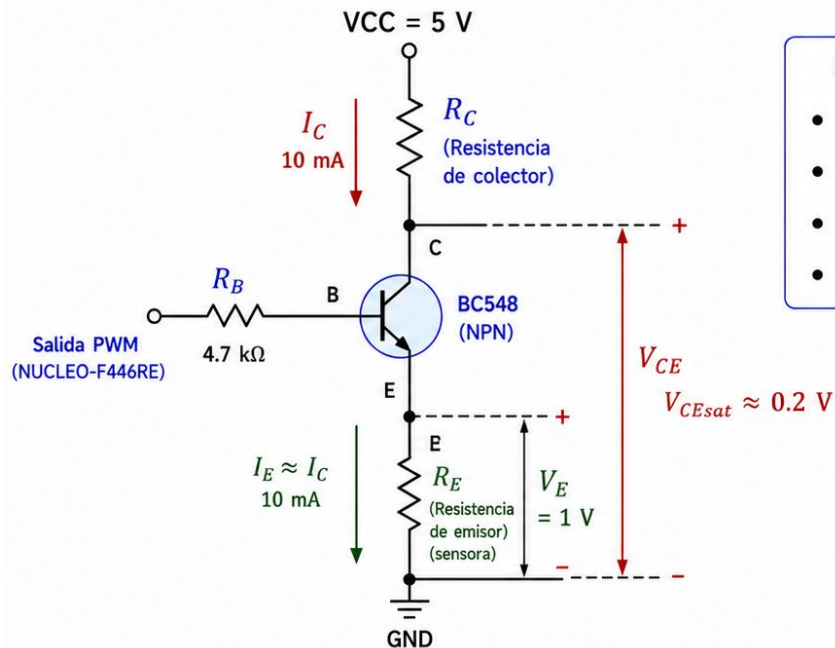
$1 \text{ V} \Rightarrow 1240$ cuentas aprox.

$0.5 \text{ V} \Rightarrow 620$ cuentas aprox.



R_E convierte **corriente** en **tensión** medible.

Cálculo de RC



Datos de diseño

- $V_{CC} = 5 \text{ V}$
- $V_{CEsat} \approx 0.2 \text{ V}$
- $V_E = 1 \text{ V}$
- $I_C = 10 \text{ mA}$

Ecuación de malla (colector-emisor)

$$V_{RC} = V_{CC} - V_{CEsat} - V_E$$



Cálculo de V_{RC}

$$V_{RC} = 5 \text{ V} - 0.2 \text{ V} - 1 \text{ V} = 3.8 \text{ V}$$



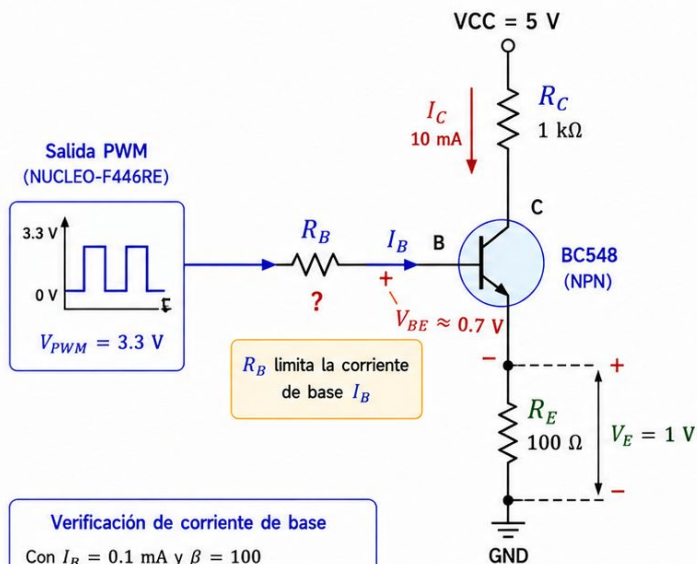
Cálculo de R_C

$$R_C = \frac{V_{RC}}{I_C} = \frac{3.8 \text{ V}}{10 \text{ mA}} = 380 \Omega$$



R_C limita la corriente que circula por el colector.

Cálculo de R_B



Verificación de corriente de base

Con $I_B = 0.1 \text{ mA}$ y $\beta = 100$
 $\Rightarrow I_C = \beta \cdot I_B = 100 \cdot 0.1 \text{ mA} = 10 \text{ mA}$
(coincide con el valor deseado)

Datos de diseño

- $V_{PWM} = 3.3 \text{ V}$
- $V_{BE} \approx 0.7 \text{ V}$
- $V_E = 1 \text{ V}$
- $I_C = 10 \text{ mA}$
- $\beta \approx 100$

1) Tensión en R_B

$$V_{RB} = V_{PWM} - V_{BE} - V_E$$
$$V_{RB} = 3.3 \text{ V} - 0.7 \text{ V} - 1 \text{ V} = 1.6 \text{ V}$$



2) Corriente de base I_B

$$I_B = \frac{I_C}{\beta}$$
$$I_B = \frac{10 \text{ mA}}{100} = 0.1 \text{ mA}$$



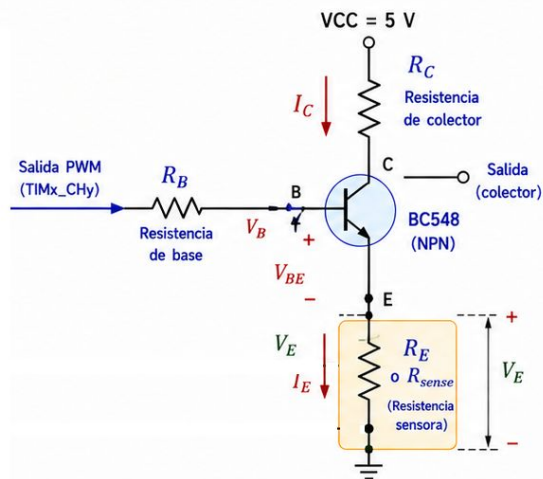
3) Resistencia R_B

$$R_B = \frac{V_{RB}}{I_B}$$
$$R_B = \frac{1.6 \text{ V}}{0.1 \text{ mA}} = 16 \text{ k}\Omega$$



R_B limita la corriente que entrega el pin PWM.

De la polarización al control con realimentación



Polarización (ejemplo)

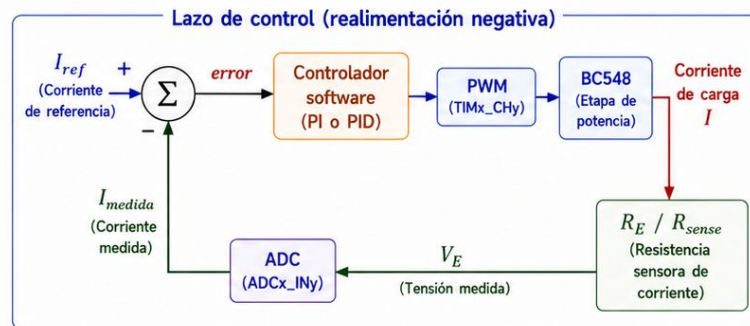
$V_{CC} = 5 \text{ V}$ $I_C \approx 10 \text{ mA}$
 $V_E = 1 \text{ V}$ $V_{BE} \approx 0.7 \text{ V}$
 $R_E = 100 \Omega$ $R_C \approx 380 \Omega$
 $R_B \approx 16 \text{ k}\Omega$

Relaciones fundamentales

$$V_E = I \cdot R_E$$

$$I_{medida} = \frac{V_E}{R_E}$$

$$error = I_{ref} - I_{medida}$$



Cálculo de la corriente y del error

$$V_E = I \cdot R_E \quad \left| \quad I_{medida} = \frac{V_E}{R_E} \quad \left| \quad error = I_{ref} - I_{medida} \right. \right.$$

Decisiones del controlador

- Si $I_{medida} < I_{ref}$ → $error > 0$ → Aumentar PWM
- Si $I_{medida} > I_{ref}$ → $error < 0$ → Disminuir PWM
- Si $I_{medida} \approx I_{ref}$ → $error \approx 0$ → Mantener PWM



La polarización fija el punto seguro; la realimentación corrige la corriente.

Encuesta clase a clase